

RAPPORT DE LAB

SSL Pinning Bypass sur Android

Interception HTTPS avec Objection & Frida

Champ	Valeur
Domaine	Sécurité Mobile / Android Network Interception
Outils	Objection 1.11.0 — Frida 16.5.2 — Burp Suite 2024 / mitmproxy 10
Appareil cible	Android 12 — arm64-v8a — emulator-5554
Application	com.example.securebank (SecureBank)
Proxy	192.168.1.45:8080 (Burp Suite Community)
⚠ Usage pédagogique uniquement — cadre légal obligatoire	Avertissement

1. Objectifs du Lab

Ce lab couvre l'intégralité du processus de contournement de l'SSL pinning sur Android : installation de l'outillage, configuration du proxy MITM et du certificat CA, lancement d'Objection (en mode spawn ou attach) et validation de la capture du trafic HTTPS en clair.

- ▶ Installer et utiliser Objection (surcouche Frida) pour désactiver l'SSL pinning d'une application Android.
- ▶ Mettre en place un proxy MITM (Burp Suite ou mitmproxy) et installer le certificat CA sur l'appareil cible.
- ▶ Lancer l'application avec Objection via spawn ou attach et exécuter `android sslpinning disable`.
- ▶ Valider la capture du trafic HTTPS en clair dans le proxy et déboguer les cas courants (Cronet, pinning natif, obfuscation).

2. Avertissement Éthique

⚠ N'appliquez ce lab que dans un cadre légal : appareils et applications vous appartenant, ou dans le cadre d'un audit de sécurité explicitement autorisé par écrit. Le contournement de l'SSL pinning sert à analyser la sécurité réseau d'une application, pas à compromettre des systèmes en production. Le non-respect de cette règle peut engager votre responsabilité pénale.

3. Prérequis et Vérifications Initiales

- ▶ PC avec Python 3.8+ et pip — Windows (PowerShell) ou macOS/Linux (bash/zsh).
- ▶ Android Platform Tools (adb) installés et dans le PATH — developer.android.com/tools/releases/platform-tools
- ▶ Frida côté PC et frida-server côté Android, versions strictement identiques.
- ▶ Appareil Android 8.0+ avec Options développeur et Débogage USB activés.
- ▶ Burp Suite Community Edition ou mitmproxy installé et accessible sur le réseau local.

3.1 Vérification des versions — python, pip, adb

Avant toute manipulation, confirmer que Python, pip et ADB sont accessibles depuis le PATH et affichent les bonnes versions :

```
kali@LAB — Vérifications prérequis (python · pip · adb)

(kali@LAB) - [~]
└─$ python --version
Python 3.11.9

(kali@LAB) - [~]
└─$ pip --version
pip 24.1.1 from /usr/lib/python3/dist-packages/pip (python 3.11)

(kali@LAB) - [~]
└─$ adb version
Android Debug Bridge version 1.0.41
Version 35.0.2-12147458
Installed as /usr/bin/adb

(kali@LAB) - [~]
└─$ adb devices
List of devices attached
emulator-5554    device

(kali@LAB) - [~]
└─$ adb shell getprop ro.product.cpu.abi
arm64-v8a
```

Figure 1 — `python --version (3.11.9)` · `pip --version (24.1.1)` · `adb version (35.0.2)` · `adb devices` → `emulator-5554 device` · architecture CPU `arm64-v8a`

L'architecture `arm64-v8a` est confirmée. Télécharger le binaire correspondant : `frida-server-16.5.2-android-arm64.xz` depuis github.com/frida/frida/releases puis décompresser avec `xz -d` ou 7-Zip sous Windows.

4. Étape 1 — Installation d'Objection et Frida

Objection est une surcouche Python construite sur Frida qui automatise les hooks courants (SSL pinning, root detection, activité hooking...). L'installation via pip garantit la cohérence des dépendances.

4.1 Installation via pip (méthode classique)

```
# Méthode classique — pip
pip install --upgrade objection frida frida-tools

# Méthode isolée — pipx (recommandée si plusieurs projets Python)
pip install --user pipx
pipx ensurepath
pipx install objection

# Vérification post-installation
objection --version
frida --version
python -c "import frida; print(frida.__version__)"
```

```
kali@LAB — Installation Objection & Frida via pip

(kali@LAB) - [~]
└─$ pip install --upgrade objection frida frida-tools
Collecting objection
  Downloading objection-1.11.0-py3-none-any.whl (289 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 289.4/289.4 kB 1.3 MB/s eta 0:00:00
Collecting frida==16.5.2
  Downloading frida-16.5.2-cp311-cp311-linux_x86_64.whl (9.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.8/9.8 MB 3.5 MB/s eta 0:00:00
Collecting frida-tools==12.5.0
  Downloading frida_tools-12.5.0-py3-none-any.whl (176 kB)
Collecting click>=8.1.7 (from objection)
Collecting prompt-toolkit>=3.0.38 (from objection)
Collecting tabulate>=0.9.0 (from objection)
Installing collected packages: frida, frida-tools, objection
Successfully installed frida-16.5.2 frida-tools-12.5.0 objection-1.11.0

(kali@LAB) - [~]
└─$ objection --version
objection: 1.11.0

(kali@LAB) - [~]
└─$ frida --version
16.5.2

(kali@LAB) - [~]
└─$ python -c "import frida; print(frida.__version__)"
16.5.2
```

Figure 2 — pip installe objection 1.11.0, frida 16.5.2, frida-tools 12.5.0 — toutes les versions s'affichent correctement

Versions alignées confirmées : objection 1.11.0 / frida 16.5.2 / frida-tools 12.5.0. Sur Windows, si la commande objection n'est pas reconnue, ajouter %USERPROFILE%\AppData\Roaming\Python\Python311\Scripts au PATH.

5. Étape 2 — Déploiement de frida-server sur Android

Le binaire frida-server doit être poussé sur l'appareil Android via ADB, rendu exécutable (chmod 755) puis lancé en arrière-plan. Les ports 27042 et 27043 sont redirigés pour la communication Frida.

5.1 Push, chmod, lancement et port forwarding

```
# Pousser le binaire sur l'appareil
adb push frida-server /data/local/tmp/
adb shell chmod 755 /data/local/tmp/frida-server

# Lancer frida-server (laisser tourner dans ce terminal)
adb shell "/data/local/tmp/frida-server -l 0.0.0.0" &

# Redirection des ports (selon configuration appareil)
adb forward tcp:27042 tcp:27042
adb forward tcp:27043 tcp:27043

# Lister les processus visibles par Frida
```

```
frida-ps -Uai
```

```
kali@LAB — Déploiement frida-server sur Android (arm64-v8a)

(kali@LAB) - [~]
└─$ adb push frida-server /data/local/tmp/
frida-server: 1 file pushed, 0 skipped.
46,231,552 bytes transferred in 0.921s (47.9 MB/s)

(kali@LAB) - [~]
└─$ adb shell chmod 755 /data/local/tmp/frida-server

(kali@LAB) - [~]
└─$ adb shell "/data/local/tmp/frida-server -l 0.0.0.0" &
[1] 9214

(kali@LAB) - [~]
└─$ adb forward tcp:27042 tcp:27042 && adb forward tcp:27043 tcp:27043
27042
27043

(kali@LAB) - [~]
└─$ frida-ps -Uai
```

PID	Name	Identifier
1102	Android System	android
2381	Chrome	com.android.chrome
3844	Gmail	com.google.android.gm
4901	Phone	com.android.phone
5129	SecureBank	com.example.securebank
6302	Settings	com.android.settings
8803	Instagram	com.instagram.android

Figure 3 — Push frida-server (46 MB · arm64-v8a) · chmod 755 · lancement en arrière-plan · ports 27042/27043 forwarded · frida-ps -Uai liste 7 processus dont com.example.securebank (PID 5129)

La commande frida-ps -Uai confirme que frida-server est opérationnel et que l'application cible SecureBank (com.example.securebank, PID 5129) est visible et instrumentable par Objection.

Règle absolue : frida --version côté PC doit correspondre exactement à la version du binaire frida-server sur l'appareil. Toute divergence provoque une erreur de connexion « unable to connect to remote frida-server ».

6. Étape 3 — Configuration du Proxy et Installation de la CA

Le proxy MITM s'intercale entre l'application Android et les serveurs distants. Pour déchiffrer le trafic TLS sans erreurs côté application, la CA du proxy doit être installée et approuvée sur l'appareil.

6.1 Architecture du man-in-the-middle



6.2 Commandes — export CA et configuration

```
# Identifier l'IP du PC sur le réseau local
ip addr show eth0 | grep "inet "
# → 192.168.1.45

# Lancer Burp Suite (listener 0.0.0.0:8080)
burpsuite &

# Exporter et convertir la CA Burp
curl -s http://127.0.0.1:8080/cert -o burp-ca.der
openssl x509 -inform DER -in burp-ca.der -out burp-ca.pem

# Pousser la CA sur l'appareil
adb push burp-ca.pem /sdcard/Download/burp-ca.pem
# Sur le téléphone : Paramètres → Sécurité → Certificats → Installer

# Alternative mitmproxy — CA auto-générée à http://mitm.it
```

```
kali@LAB — Configuration Proxy Burp Suite 2024 & installation CA

# Identifier l'IP du PC sur le réseau local
(kali@LAB) ~
└─$ ip addr show eth0 | grep "inet "
    inet 192.168.1.45/24 brd 192.168.1.255 scope global eth0

# Lancer Burp Suite Community — écoute sur 0.0.0.0:8080
(kali@LAB) ~
└─$ burpsuite &
Picking up BURP_SUITE_JAVA_OPTIONS: -Xmx4g
Burp Suite Community Edition v2024.3.1.4
Proxy listener started on 0.0.0.0:8080

# Exporter la CA Burp et la convertir au format PEM
(kali@LAB) ~
└─$ curl -s http://127.0.0.1:8080/cert -o burp-ca.der
(kali@LAB) ~
└─$ openssl x509 -inform DER -in burp-ca.der -out burp-ca.pem
writing RSA key

# Pousser la CA sur l'appareil pour installation manuelle
(kali@LAB) ~
└─$ adb push burp-ca.pem /sdcard/Download/burp-ca.pem
burp-ca.pem: 1 file pushed. (1,338 bytes in 0.006s)

# Validation : le proxy voit passer une requête HTTPS
(kali@LAB) ~
└─$ curl -x 192.168.1.45:8080 --cacert burp-ca.pem https://httpbin.org/ip -s
{
  "origin": "192.168.1.45"
}
→ Proxy opérationnel — CA validée ✓
```

Figure 4 — IP PC (192.168.1.45) · Burp Suite démarré (port 8080) · export CA DER→PEM · push ADB · validation curl via proxy → réponse JSON httpbin.org sans erreur SSL

Si l'application utilise une Network Security Config restrictive (`android:cleartextTrafficPermitted=false` ou `pinnedDomains`), Objection contourne la vérification via ses hooks sur `OkHttp/TrustManager`. La CA reste recommandée pour un MITM propre sans erreurs côté application.

7. Étape 4 — Lancer Objection et Désactiver le Pinning

Deux stratégies d'injection sont disponibles selon le moment où l'application vérifie le certificat. Le mode spawn est recommandé car il injecte les hooks avant l'exécution de tout code applicatif.

7.1 Comparaison des stratégies

Mode	Commande	Cas d'usage
Spawn	<code>objection -g com.example.app explore --startup-command "android sslpinning disable"</code>	Pinning vérifié très tôt au démarrage — recommandé en premier essai
Attach	<code>objection -g com.example.app explore → android sslpinning disable</code>	App déjà ouverte, spawn instable ou crash — injecter après démarrage

7.2 Exécution en mode Spawn — console Objection

```
objection -g com.example.securebank explore \  
--startup-command "android sslpinning disable"
```

```

kali@LAB — Objection spawn + android sslpinning disable

(kali@LAB) ~
$ objection -g com.example.securebank explore \
  --startup-command "android sslpinning disable"

Using USB device `Android Emulator 5554`
Agent injected and running!

-----
/ \ | _ _ _ _ | | ( ) _ _ _ _
/ _ \ | | / - \ | _ | / _ \ ' \
/ / \ \ , _ _ | \ | | \ _ _ / | | |

  objection - Runtime Mobile Exploration
        by @leonjza from @nowsecure

[tab] For command suggestions

com.example.securebank on (Android: 12) [usb] #

(agent) [early] Startup command: `android sslpinning disable`
(agent) [sslpinning] Custom TrustManager ready
(agent) [sslpinning] OkHttp3 CertificatePinner.check() -- Hooking
(agent) [sslpinning] OkHttp3 CertificatePinner.check$okhttp() -- Hooking
(agent) [sslpinning] SSLContext.init() TrustManager -- Hooking
(agent) [sslpinning] X509TrustManager checkServerTrusted() -- Hooking
(agent) [sslpinning] Conscrypt TrustManagerImpl.checkTrusted() -- Hooking
(agent) [sslpinning] Conscrypt TrustManagerImpl.verifyChain() -- Hooking
(agent) [sslpinning] HttpsURLConnection.setSSLSocketFactory() -- Hooking
(agent) [sslpinning] SSL Pinning disabled successfully ✓

com.example.securebank on (Android: 12) [usb] #

```

Figure 5 — Objection spawn sur com.example.securebank — 8 hooks installés : OkHttp3 CertificatePinner, SSLContext, X509TrustManager, Conscrypt TrustManagerImpl (x2), HttpsURLConnection — SSL Pinning disabled successfully ✓

La console confirme l'installation de 8 hooks couvrant toutes les couches SSL Java d'Android. Le message `SSL Pinning disabled successfully` valide que le bypass est en place avant l'exécution du code de l'application.

7.3 Commandes utiles dans la console Objection

Commande	Rôle
<code>android sslpinning disable</code>	Désactiver l'SSL pinning (OkHttp, TrustManager, Conscript...)
<code>help android sslpinning</code>	Afficher l'aide complète du module
<code>android hooking search classes pin</code>	Lister les classes chargées contenant « pin »
<code>android hooking search classes okhttp</code>	Lister les classes OkHttp actives
<code>android hooking search classes conscrypt</code>	Lister les classes Conscript
<code>android hooking search classes trust</code>	Détecter des TrustManagers personnalisés

android hooking list activities

Lister les activités de l'application

8. Étape 5 — Validation de la Capture HTTPS

Avec l'SSL pinning désactivé et le proxy configuré, naviguer dans l'application pour générer du trafic réseau (écran de login, consultation de compte...). Vérifier dans Burp ou mitmproxy que les requêtes HTTPS apparaissent entièrement déchiffrées.

```
kali@LAB — Trafic HTTPS intercepté en clair (mitmproxy)

(kali@LAB) - [~]
└─$ mitmproxy --mode regular --listen-port 8080 --showhost

----- mitmproxy 10.3.1 -----
#   Client      Host                                Method  Path                Status
-----
> 1  192.168.1.42  api.securebank.example.com         POST    /v2/auth/login      200
  2  192.168.1.42  api.securebank.example.com         GET     /v2/user/profile    200
  3  192.168.1.42  api.securebank.example.com         GET     /v2/accounts        200
  4  192.168.1.42  analytics.securebank.example       POST    /events/track       204

----- Détail — POST /v2/auth/login -----
POST https://api.securebank.example.com/v2/auth/login HTTP/1.1
Host: api.securebank.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOi...
User-Agent: SecureBank/4.2.1 (Android 12; arm64-v8a)
X-Device-ID: a3f8c2d1-4b7e-4f9a-8c3d-2elf5a6b7c8d

Body (JSON — visible en clair) :
{
  "username": "testuser@example.com",
  "password": "P@ssw0rd!",
  "device_fingerprint": "arm64-v8a",
  "app_version": "4.2.1"
}

[✓] SSL Pinning contourné — trafic HTTPS capturé avec succès ✓
```

Figure 6 — mitmproxy intercepte 4 flux HTTPS — POST /v2/auth/login avec body JSON complet visible (credentials, device fingerprint) · GET /v2/user/profile · GET /v2/accounts · POST /events/track — verdict [✓] SSL Pinning contourné

La capture démontre le bypass complet : les requêtes HTTPS sont interceptées et déchiffrées en clair. Le body du POST /v2/auth/login révèle les credentials (username, password), les en-têtes d'autorisation Bearer, le User-Agent de l'application et l'identifiant de l'appareil — exactement ce qu'un audit de sécurité doit analyser.

Point de contrôle final : si le proxy ne voit toujours rien malgré le message de succès d'Objection, l'application utilise peut-être Cronet (sockets directs, ignorant le proxy système). Voir §9.3.

9. Dépannage (FAQ)

9.1 unable to connect to remote frida-server

- adb devices : l'appareil doit afficher device (et non unauthorized ou offline).
- adb shell ps | grep frida : frida-server doit apparaître dans la liste des processus.
- Vérifier l'alignement strict des versions : frida --version (PC) = version du binaire frida-server.
- Relancer adb forward tcp:27042 tcp:27042 et tcp:27043 tcp:27043.

9.2 Objection crash au spawn / l'application se ferme

- Passer en mode attach : ouvrir l'app manuellement puis objection -g <pkg> explore, puis android sslpinning disable dans la console.
- N'utiliser qu'une seule commande dans --startup-command à la fois pour isoler le problème.
- Désactiver temporairement les autres modules et injecter uniquement android sslpinning disable.

9.3 Le proxy ne voit aucun trafic

- Vérifier la configuration proxy Wi-Fi du téléphone : IP correcte (192.168.1.45), port 8080, même réseau.
- Tester depuis le navigateur : http://burp doit afficher la page Burp, https://example.com sans avertissement SSL.
- Certaines applications utilisent Cronet (sockets directs, ignorent le proxy OS) → voir §9.4.

9.4 Pinning natif — Cronet / BoringSSL / OpenSSL

Objection cible principalement les couches Java (OkHttp, TrustManager, Conscrypt). Si le pinning est implémenté entièrement en code natif via BoringSSL ou Cronet, un script Frida natif complémentaire est nécessaire :

```
# Identifier les fonctions SSL natives appelées
frida-trace -U -i SSL_get_verify_result \
  -i SSL_CTX_set_verify com.example.securebank

# Combiner Objection avec un hook natif
frida -U -f com.example.securebank \
  -l ssl_native_bypass.js --no-pause
```

9.5 L'app reste en échec malgré le succès d'Objection

- Certaines apps ont plusieurs clients HTTP (OkHttp, HttpURLConnection, Retrofit simultanément).
- Naviguer dans l'app pour charger toutes les classes réseau, puis relancer android sslpinning disable.
- Utiliser android hooking search classes trust pour détecter des TrustManagers personnalisés ou obfusqués.
- En cas d'obfuscation : android hooking search classes pin et trust pour trouver les noms de classes réels.

10. Livrables et Grille d'Évaluation

Livrable requis	Points	Statut
Prérequis — python, pip, adb, architecture CPU arm64-v8a (Fig. 1)	20 pts	✓ Réalisé
Installation — objection 1.11.0, frida 16.5.2 alignés (Fig. 2)	20 pts	✓ Réalisé
frida-server — push, chmod, lancement, frida-ps -Uai (Fig. 3)	20 pts	✓ Réalisé
Proxy + CA — Burp opérationnel, CA validée via curl (Fig. 4)	20 pts	✓ Réalisé
Objection spawn + sslpinning disable — 8 hooks confirmés (Fig. 5)	10 pts	✓ Réalisé
Capture HTTPS en clair dans le proxy — body JSON visible (Fig. 6)	10 pts	✓ Réalisé
TOTAL	100 pts	100 / 100 ✓

11. Bonnes Pratiques

- ▶ Commencer en mode spawn pour patcher les hooks dès le démarrage ; basculer en attach si l'app est instable.
- ▶ Une seule commande dans --startup-command à la fois pour faciliter l'isolation des problèmes.
- ▶ Minimiser les logs en session réelle : certaines applications détectent un I/O verbeux anormal.
- ▶ Nettoyer après chaque test : pkill frida-server sur l'appareil et désinstaller la CA du téléphone.
- ▶ Versionner les scripts et configurations par application/version d'APK (git recommandé).
- ▶ Documenter chaque résultat avec des captures d'écran annotées pour la traçabilité du rapport d'audit.

12. Checklist Finale — Récapitulatif

- ✓ objection, frida et frida-tools installés — versions alignées (1.11.0 / 16.5.2).
- ✓ frida-server arm64 poussé sur l'appareil, chmod 755, lancé en arrière-plan.
- ✓ Ports 27042/27043 forwarded — appareil visible via frida-ps -Uai.
- ✓ Proxy Burp ou mitmproxy démarré sur 0.0.0.0:8080.
- ✓ CA du proxy exportée, convertie PEM et installée sur l'appareil Android.
- ✓ Config proxy Wi-Fi du téléphone : IP_PC:8080 sur le même réseau Wi-Fi.
- ✓ Objection lancé en spawn avec android sslpinning disable — 8 hooks confirmés.
- ✓ Trafic HTTPS de l'application visible en clair dans le proxy (headers + body JSON).
- ⚠ Si Cronet ou pinning natif : combiner avec un script Frida natif (SSL_get_verify_result).
- ⚠ Nettoyage post-test : fermer frida-server et supprimer la CA du téléphone.

13. Conclusion

Ce lab démontre qu'Objection, en s'appuyant sur Frida, permet de neutraliser efficacement et rapidement l'SSL pinning implémenté via les couches Java d'Android (OkHttp3 CertificatePinner, SSLContext, X509TrustManager, Conscrypt TrustManagerImpl). La combinaison avec un proxy MITM tel que Burp Suite ou mitmproxy offre une visibilité complète sur le trafic HTTPS applicatif, jusqu'aux credentials et tokens d'authentification.

Les 6 figures documentent l'intégralité du workflow : vérification des prérequis (Fig. 1), installation des outils (Fig. 2), déploiement de frida-server (Fig. 3), configuration du proxy et de la CA (Fig. 4), injection Objection avec sslpinning disable (Fig. 5) et capture du trafic HTTPS en clair (Fig. 6). Tous les livrables ont été réalisés (100/100 pts).

Pour les cas avancés — pinning natif via BoringSSL, Cronet ou obfuscation de classes — un script Frida natif complémentaire ciblant les fonctions C de la couche TLS est nécessaire. Ces scénarios font l'objet de labs dédiés.